

OpenMower-TAF-ROS - Implementierungsdokumentation - GPS-State-MQTT

Version v0.1 - 2026-06-23

Diese Dokumentation beschreibt die umgesetzte Erweiterung im ROS-/MQTT-Backend, mit der die aktuell verfügbaren GPS-Satellitendaten aus ROS als separater MQTT-Hauptbereich `gps_state` bereitgestellt werden. Die App kann diese Daten später ohne eigene ROS-Anbindung abonnieren und visualisieren.

Angabe	Wert
Projektname	OpenMower-TAF-ROS
Dokumentenart	Implementierungsdokumentation / MQTT-Schnittstelle
Thema	GPS-State-MQTT
Erstellungsdatum	2026-06-23
Änderungsdatum	2026-06-23
Version	v0.1
Status	Entwurf
Autor / verantwortliche Person	ChatGPT
Ablageort	Codepaket-ZIP, Dokumentation, PDF und DOCX
Referenzen	OpenMower-TAF-ROS, xbot_monitoring, xbot_driver_gps, GPS-State-MQTT-App-Spezifikation

1. Ziel der Änderung

Ziel ist eine getrennte MQTT-Bereitstellung der aktuellen Satellitenlage. Die Daten werden nicht in die normale Sensorseite integriert, sondern als eigenständiger MQTT-Bereich `gps_state` veröffentlicht. Damit bleibt die große Satellitenliste von den allgemeinen Sensorwerten getrennt.

2. Geänderte Dateien

Datei	Änderung
src/lib/xbot_monitoring/src/xbot_monitoring.cpp	Erweiterung um GPS-State-Subscriber, State1-State4-Payloads, <code>gps_state/settings/json</code> , MQTT-Settings-Kommandos, Validierung, Publish-Rate und Persistenz.
_Dokumentation/2026-06-23 - OpenMower-TAF-ROS - Implementierungsdokumentation - GPS-State-MQTT - v0.1.docx	Bearbeitbare Dokumentation der Änderung.
_Dokumentation/2026-06-23 - OpenMower-TAF-ROS - Implementierungsdokumentation - GPS-State-MQTT - v0.1.pdf	PDF-Ausgabe der Dokumentation.
commit_message.txt	Englische Commit-Nachricht mit Datum, Zusammenfassung und Dateiliste.

3. Technische Umsetzung

Baustein	Umsetzung
ROS-Quelle	Subscriber auf <code>/ll/position/gps/satellites</code> mit Message-Typ <code>xbot_msgs/GnssSatelliteArray</code> .
MQTT-Publisher	Veröffentlichung über die bestehende <code>try_publish</code> -Funktion in <code>xbot_monitoring</code> . Externe MQTT-Prefixe werden dadurch wie bisher berücksichtigt.
Datenmodell	Vier nicht additive, jeweils eigenständig nutzbare Payloads: <code>gps_state/state1</code> bis <code>gps_state/state4</code> .
Metadaten	Einmalige Beschreibung der States und Settings über <code>gps_state/settings/json</code> im <code>settings_v2</code> -Stil.
Steuerung	App-Kommandos über <code>gps_state/settings/set/renew/json</code> , <code>gps_state/settings/set/session/json</code> und <code>gps_state/settings/set/persistent/json</code> .
Persistenz	Persistente Werte werden im Namespace <code>gps_state</code> in der vorhandenen <code>settings_persistent.json</code> abgelegt.

4. MQTT-Topicstruktur

Topic	Richtung	Retained	Beschreibung
gps_state/state1	Backend -> App	Ja	Kompakter GPS-Zustand mit Verfügbarkeit, Qualität, visible, used, avg_cn0 und updated_at.
gps_state/state2	Backend -> App	Ja	Erweiterte Zusammenfassung mit min/max C/N0, weak/good count und Systemverteilung.
gps_state/state3	Backend -> App	Nein	Liste der aktuell für den Fix verwendeten Satelliten plus Basiswerte.
gps_state/state4	Backend -> App	Nein	Vollständige Satellitenliste, standardmäßig deaktiviert und primär für Debug/Expertenansicht.
gps_state/settings/json	Backend -> App	Ja	Metadaten, Labels, Descriptions, Settings und aktuelle Werte.
gps_state/settings/set/renew/json	App -> Backend	Nein	Fordert Settings und zuletzt bekannte State-Payloads neu an.
gps_state/settings/set/session/json	App -> Backend	Nein	Ändert GPS-State-Einstellungen nur für die laufende Session.
gps_state/settings/set/persistent/json	App -> Backend	Nein	Speichert GPS-State-Einstellungen persistent.
gps_state/settings/validation/json	Backend -> App	Nein	Validierungsantwort auf Session- und Persistent-Änderungen.

5. State-Inhalte

State	Topic	Inhalt
State1	gps_state/state1	available, quality, visible, used, avg_cn0, updated_at.
State2	gps_state/state2	State1-Basiswerte plus sensor_stamp, min_cn0, max_cn0, weak_count, good_count und systems.
State3	gps_state/state3	State1-Basiswerte plus sensor_stamp, min_cn0, max_cn0 und satellites-Liste mit USED=true.
State4	gps_state/state4	State1-Basiswerte plus sensor_stamp, min_cn0, max_cn0 und satellites-Liste mit allen sichtbaren Satelliten.

6. Einstellbare Werte

Setting	Typ	Default	Bereich	Beschreibung
enabled	bool	true	-	Aktiviert oder deaktiviert die GPS-State-Ausgabe.
publish_rate_hz	double	1.0	0.1 bis 5.0 Hz	Maximale MQTT-Veröffentlichungsrate.
publish_state1	bool	true	-	Veröffentlicht <code>gps_state/state1</code> .
publish_state2	bool	true	-	Veröffentlicht <code>gps_state/state2</code> .
publish_state3	bool	true	-	Veröffentlicht <code>gps_state/state3</code> .
publish_state4	bool	false	-	Veröffentlicht <code>gps_state/state4</code> ; standardmäßig aus wegen Datenmenge.
weak_cn0_threshold	double	20.0	0.0 bis 60.0 dB-Hz	Schwelle für schwache verwendete Satelliten.
good_cn0_threshold	double	30.0	0.0 bis 60.0 dB-Hz	Schwelle für gute verwendete Satelliten.

7. Beispiel-Kommandos für MQTT

Status neu anfordern:

```
mosquitto_pub -h localhost -t 'gps_state/settings/set/renew/json' -m '{}'
```

Publish-Rate live auf 0.5 Hz setzen:

```
mosquitto_pub -h localhost -t 'gps_state/settings/set/session/json' -m '{"publish_rate_hz":{"value":0.5}}'
```

Publish-Rate dauerhaft auf 1.0 Hz speichern:

```
mosquitto_pub -h localhost -t 'gps_state/settings/set/persistent/json' -m '{"publish_rate_hz":{"value":1.0}}'
```

State4 temporär aktivieren:

```
mosquitto_pub -h localhost -t 'gps_state/settings/set/session/json' -m '{"publish_state4":{"value":true}}'
```

Alle GPS-State-Topics beobachten:

```
mosquitto_sub -h localhost -t 'gps_state/#' -v
```

8. Validierung und Persistenz

Session- und Persistent-Payloads müssen ein JSON-Objekt sein. Bekannte Settings werden typ- und bereichsgeprüft.

Unbekannte Keys werden abgelehnt. Bei persistenten Änderungen schreibt `xbot_monitoring` die akzeptierten persistent-Werte in den Namespace `gps_state` der vorhandenen `settings_persistent.json`.

```
{
  "valid": true,
  "namespace": "gps_state",
  "mode": "session",
  "accepted": {"publish_rate_hz": ["value"]},
  "rejected": {},
  "remarks": []
}
```

9. Testplan

Test	Befehl / Prüfung	Erwartung
ROS-Quelle prüfen	rostopic echo -n 1 //position/gps/satellites	GnssSatelliteArray mit satellites[] erscheint.
MQTT-Livewerte prüfen	mosquitto_sub -h localhost -t gps_state/# -v	state1 bis state3 und settings/json erscheinen.
Renew prüfen	Publish auf gps_state/settings/set/renew/json	settings/json und letzte States werden erneut gesendet.
Session-Rate prüfen	publish_rate_hz per session auf 0.5 setzen	Validierung valid=true; Ausgabe verlangsamt sich.
State4 prüfen	publish_state4 per session aktivieren	gps_state/state4 wird mit vollständiger Satellitenliste gesendet.
Persistent prüfen	publish_rate_hz per persistent speichern	settings_persistent.json enthält gps_state.publish_rate_hz.persistent.

10. Änderungshistorie

Version	Datum	Bearbeiter	Status	Änderung
v0.1	2026-06-23	ChatGPT	Entwurf	Erste Implementierung der separaten GPS-State-MQTT-Ausgabe in xbot_monitoring mit Dokumentation.

11. Kurzfazit

Die Änderung stellt GPS-Satellitendaten als separaten MQTT-Bereich bereit. Die App kann die vier States unabhängig abonnieren. Die Ausgabe ist über settings_v2-Metadaten beschrieben, über MQTT steuerbar und mit 1 Hz standardmäßig ressourcenschonend begrenzt.